

# Documento 05 – Arquitetura do Sistema

**Projeto:** Sistema de Gestão de Territórios e Publicações (GTP)

**Versão:** 1.0.0

**Status:** Em Elaboração

**Data:** Julho/2026

**Autor:** Fabio André Zatta

## Capítulo 1 – Introdução

### 1.1 Objetivo

Este documento define a arquitetura de software do Gestor de Territórios e Publicações (GTP). Seu objetivo é estabelecer a organização estrutural da solução, os princípios arquiteturais, as tecnologias adotadas, os padrões de desenvolvimento e as diretrizes que orientarão a implementação, manutenção e evolução do sistema. A arquitetura foi projetada para atender aos requisitos funcionais e não funcionais identificados nos Documentos 00 a 04, oferecendo uma solução moderna, modular, segura e escalável.

### 1.2 Escopo

Este documento descreve:

- Arquitetura geral do sistema;
- Organização em camadas;
- Componentes principais;
- Fluxo de comunicação entre frontend e backend;
- Padrões arquiteturais;
- Tecnologias adotadas;
- Estratégias de segurança;
- Integração entre módulos;
- Persistência de dados;
- Infraestrutura de execução;
- Diretrizes para evolução do sistema.

Os detalhes específicos de backend, frontend, segurança, Docker e API REST serão aprofundados nos Documentos 06 a 10.

### 1.3 Objetivos Arquiteturais

A arquitetura do GTP foi definida para atender aos seguintes objetivos:

**Modularidade:** O sistema será dividido em módulos independentes, reduzindo acoplamento e facilitando manutenção e evolução.

**Escalabilidade:** A solução deverá suportar crescimento gradual, tanto em funcionalidades quanto em volume de dados e usuários.

**Manutenibilidade:** A estrutura do código deverá facilitar correções, testes e implementação de novas funcionalidades.

**Segurança:** Todas as operações serão protegidas por autenticação, autorização baseada em papéis (RBAC), auditoria e mecanismos de proteção de dados.

**Desempenho:** A arquitetura deverá minimizar tempos de resposta e otimizar o acesso aos dados.

**Testabilidade:** A organização do sistema permitirá testes unitários, de integração e funcionais de forma simples e automatizada.

## 1.4 Princípios Arquiteturais

O desenvolvimento do GTP seguirá os seguintes princípios:

- Separação de responsabilidades;
- Baixo acoplamento;
- Alta coesão;
- Código limpo (Clean Code);
- Arquitetura orientada ao domínio;
- Desenvolvimento orientado por testes (quando aplicável);
- Segurança desde a concepção (Security by Design);
- Observabilidade;
- Evolução incremental.

## 1.5 Padrões Arquiteturais Adotados

O GTP combinará padrões consolidados da engenharia de software para obter uma arquitetura robusta e flexível.

**Domain-Driven Design (DDD):** O domínio do negócio será o centro da aplicação, organizado em módulos como: Congregações, Pessoas, Usuários, Territórios, Publicações, Pedidos, Campanhas e Relatórios. Cada módulo possuirá responsabilidades bem definidas.

**Clean Architecture:** A aplicação será organizada em camadas, separando regras de negócio, casos de uso, infraestrutura e interfaces. Benefícios: Independência de frameworks, facilidade para testes, baixo acoplamento e regras de negócio isoladas.

**Arquitetura em Camadas:** A organização geral seguirá a estrutura do Frontend (React) para a API REST (Spring Boot), passando pela Camada de Aplicação (Use Cases), Camada de Domínio (DDD) e Infraestrutura (JPA, PostgreSQL).

## 1.6 Arquitetura Geral

A solução será composta por três grandes blocos:

**Frontend:** Responsável pela interface com o usuário. Tecnologias previstas: React 19, TypeScript, Vite, Tailwind CSS, TanStack Query, Zustand.

**Backend:** Responsável pelas regras de negócio e exposição da API REST. Tecnologias previstas: Java 21, Spring Boot 3, Spring Security, JWT, Hibernate, Spring Data JPA, Flyway.

**Banco de Dados:** Responsável pelo armazenamento persistente das informações. Tecnologia prevista: PostgreSQL.

## 1.7 Visão Geral dos Componentes

O fluxo de comunicação segue do Usuário para o Frontend (React + TypeScript), via HTTPS /

REST para o Backend (Spring Boot 3), e via Spring Data JPA para o PostgreSQL.

## 1.8 Tecnologias Selecionadas

| Camada              | Tecnologia                  |
|---------------------|-----------------------------|
| Linguagem Backend   | Java 21                     |
| Framework Backend   | Spring Boot 3               |
| Segurança           | Spring Security + JWT       |
| Persistência        | Spring Data JPA / Hibernate |
| Banco de Dados      | PostgreSQL                  |
| Migrações           | Flyway                      |
| Frontend            | React 19                    |
| Linguagem Frontend  | TypeScript                  |
| Build               | Vite                        |
| Estilização         | Tailwind CSS                |
| Estado Global       | Zustand                     |
| Consumo de API      | TanStack Query              |
| Contêineres         | Docker                      |
| Versionamento       | Git                         |
| Documentação da API | OpenAPI / Swagger           |

## Capítulo 2 – Arquitetura Geral e Organização dos Componentes

### 2.1 Objetivo da Arquitetura

A arquitetura do Gestor de Territórios e Publicações (GTP) foi concebida para atender às necessidades operacionais das Congregações, proporcionando uma solução moderna, modular, segura e preparada para evolução contínua. A organização dos componentes visa separar responsabilidades de forma clara, reduzir o acoplamento entre módulos, facilitar testes e manutenção, permitir evolução incremental, garantir segurança e rastreabilidade, e simplificar futuras integrações.

### 2.2 Visão Geral da Solução

O GTP será composto por três grandes camadas tecnológicas: Usuário/Navegador, Frontend

(React 19), Backend (Spring Boot 3) e PostgreSQL 17. Cada camada possui responsabilidades bem definidas e se comunica exclusivamente por interfaces padronizadas.

## 2.3 Organização em Camadas

A arquitetura segue o princípio da separação de responsabilidades nas seguintes camadas:

- **Camada de Interface (React):** Apresentação das telas, validações iniciais, navegação, consumo da API REST e gerenciamento de sessão.
- **Camada de API (Spring Boot):** Receber requisições HTTP, validar dados de entrada, autenticar usuários, encaminhar solicitações para a camada de aplicação e devolver respostas padronizadas.
- **Camada de Aplicação (Casos de Uso):** Coordena operações, executa fluxos de negócio, controla transações e orquestra serviços de domínio (ex: Registrar Pedido, Reservar Território).
- **Camada de Domínio (Negócio):** Contém entidades, objetos de valor, agregados, regras de negócio e serviços de domínio. Sem dependência de frameworks.
- **Camada de Infraestrutura:** Acesso ao banco de dados, autenticação JWT, integração com serviços externos, logs, auditoria e Docker.

## 2.4 Arquitetura por Componentes

O sistema será dividido em componentes independentes: Autenticação, Congregações, Pessoas, Usuários, Territórios, Publicações, Pedidos, Campanhas, Notificações, Configurações, Relatórios, Auditoria, Segurança e Integrações. Cada componente possuirá API própria, serviços, regras de negócio, repositórios, DTOs e testes.

## 2.5 Organização por Módulos

**Módulo de Autenticação:** login, logout, autenticação, recuperação de senha, renovação de token, controle de sessão.

**Módulo Congregações:** cadastro, configuração, parâmetros, administração.

**Módulo Pessoas:** cadastro dos membros da Congregação (dados pessoais, contatos, situação, histórico).

**Módulo Usuários:** contas, perfis, permissões, bloqueios, autenticação.

**Módulo Territórios:** cadastro, mapas, reservas, retiradas, devoluções, histórico.

**Módulo Publicações:** estoque, entradas, saídas, inventários, movimentações.

**Módulo Pedidos:** solicitações, separação, entrega, cancelamentos, campanhas.

**Módulo Campanhas:** campanhas especiais, distribuição, acompanhamento.

**Módulo Relatórios:** dashboards, relatórios, indicadores, exportações.

**Módulo Configurações:** parâmetros, categorias, regras, preferências.

**Módulo Auditoria:** logins, alterações, exclusões, configurações, movimentações.

## 2.6 Fluxo Geral de Comunicação

Usuário → Frontend → API REST → Controller → Caso de Uso → Serviço de Domínio → Repositório → Banco de Dados → Resposta → Frontend → Usuário.

## 2.7 Organização Física do Projeto

**Backend (gtp-backend/):** src/main/java, src/main/resources, src/test.

**Frontend (gtp-frontend/):** src/assets, components, features, hooks, pages, routes, services, store, styles, types, utils.

## 2.8 Organização Lógica do Backend

Pacotes sob br.com.gtp: config, security, shared, domain, application, infrastructure, interfaces.

## 2.9 Organização Lógica do Frontend

Diretórios sob src/: app, components, features, pages, layouts, hooks, services, stores, routes, types, utils, styles.

## 2.10 Fluxo de Dados

Browser → React → Axios → Spring Boot → Use Case → JPA → PostgreSQL → JPA → JSON → React.

## 2.11 Comunicação entre Componentes

Os componentes do sistema se comunicarão exclusivamente por interfaces definidas, evitando dependências diretas. Princípios: baixo acoplamento, alta coesão, inversão de dependência e reutilização de serviços.

## 2.12 Escalabilidade

A arquitetura foi projetada para suportar novas funcionalidades, novos módulos, novas integrações, aumento de usuários, crescimento de banco de dados e futuras aplicações móveis.

# Capítulo 3 – Arquitetura do Domínio (DDD)

## 3.1 Objetivo

Definir a arquitetura do domínio do GTP. A modelagem do domínio representa o núcleo do sistema, isolando as regras de negócio das tecnologias utilizadas, facilitando manutenção e apoiando testes unitários.

## 3.2 O Domínio do GTP

Compreende a gestão integrada de Congregações, Pessoas, Usuários, Territórios, Publicações, Estoque, Pedidos, Campanhas, Auditoria, Notificações e Configurações.

## 3.3 Linguagem Ubíqua (Ubiquitous Language)

| Termo       | Definição                                                                                       |
|-------------|-------------------------------------------------------------------------------------------------|
| Congregação | Comunidade responsável pela administração dos territórios e publicações em sua área de atuação. |
| Publicador  | Membro da Congregação autorizado a utilizar territórios e solicitar publicações.                |
| Território  | Área geográfica destinada à atividade de pregação.                                              |
| Publicação  | Material controlado pelo estoque da Congregação.                                                |
| Pedido      | Solicitação de publicações realizada por um publicador ou pela Congregação.                     |
| Campanha    | Distribuição organizada de publicações em período específico.                                   |
| Estoque     | Quantidade disponível de publicações.                                                           |
| Reserva     | Solicitação de utilização futura de um território.                                              |
| Retirada    | Entrega efetiva de um território ou publicação ao responsável.                                  |
| Devolução   | Retorno de um território após sua utilização.                                                   |

### 3.4 Bounded Contexts

O GTP será dividido em contextos delimitados: Identity (Usuários, Autenticação, Permissões), Congregation (Congregações, Pessoas, Configurações), Territory (Territórios, Reservas, Devoluções), Publication (Publicações, Estoque, Inventários), Orders (Pedidos, Campanhas, Distribuição) e Administration (Auditoria, Relatórios, Notificações, Integrações).

### 3.5 Relacionamento entre os Contextos

Os contextos não devem depender diretamente uns dos outros. A comunicação ocorrerá por interfaces bem definidas e serviços de aplicação.

### 3.6 Entidades (Entities)

- **Congregação:** id, nome, código, endereço, telefone, e-mail, status.
- **Pessoa:** id, nome, data de nascimento, telefone, e-mail, endereço, situação.
- **Usuário:** id, login, senha (criptografada), perfil, status, último acesso.
- **Território:** id, código, descrição, tipo, situação, data da última devolução.
- **Publicação:** id, código, nome, categoria, idioma, quantidade em estoque.
- **Pedido:** id, número, solicitante, data, situação, itens.

- **Campanha:** id, nome, período, objetivo, situação.

### 3.7 Objetos de Valor (Value Objects)

Endereço, Telefone, E-mail, Coordenadas Geográficas, Intervalo de Datas, Quantidade, Código de Território, Código de Publicação.

### 3.8 Agregados (Aggregates)

**Aggregate Congregação:** Congregação (raiz), Pessoas, Usuários, Configurações.

**Aggregate Território:** Território (raiz), Histórico de Retiradas, Reservas, Devoluções.

**Aggregate Publicação:** Publicação (raiz), Estoque, Inventário, Movimentações.

**Aggregate Pedido:** Pedido (raiz), Itens do Pedido, Histórico, Entrega.

**Aggregate Campanha:** Campanha (raiz), Pedidos, Distribuições.

### 3.9 Serviços de Domínio (Domain Services)

Serviço de Distribuição (validar estoque, reservar quantidades), Serviço de Territórios (verificar disponibilidade, validar devolução), Serviço de Campanhas (iniciar/encerrar campanhas) e Serviço de Auditoria.

### 3.10 Repositórios (Repositories)

Abstraem o acesso aos dados: CongregacaoRepository, PessoaRepository, UsuarioRepository, TerritorioRepository, PublicacaoRepository, PedidoRepository, CampanhaRepository.

### 3.11 Fábricas (Factories)

PedidoFactory, CampanhaFactory, TerritorioFactory.

### 3.12 Eventos de Domínio (Domain Events)

PedidoCriado, PedidoEntregue, TerritorioReservado, TerritorioDevolvido, PublicacaoRecebida, EstoqueAtualizado, CampanhaIniciada, CampanhaEncerrada, UsuarioBloqueado.

### 3.13 Invariantes do Domínio

- Um território não pode ser retirado simultaneamente por dois publicadores;
- Um pedido não pode ser entregue sem estoque disponível;
- Um usuário sem permissão não pode executar operações administrativas;
- Uma campanha encerrada não pode receber novos pedidos;
- Toda movimentação de estoque deve gerar registro de auditoria.

### 3.14 Benefícios da Arquitetura de Domínio

Alinhamento entre código e negócio, isolamento das regras de negócio, melhor testabilidade e menor acoplamento.

# Capítulo 4 – Arquitetura Backend

## 4.1 Objetivo

Definir a arquitetura do backend do GTP, responsável por expor a API REST, controlar autenticação/autorização, persistir dados no PostgreSQL e registrar auditoria.

## 4.2 Stack Tecnológica

Java 21 (LTS), Spring Boot 3.x, Maven, Spring Data JPA, Hibernate, PostgreSQL 17, Flyway, Spring Security, JWT, OpenAPI/Swagger, MapStruct, Lombok, Jakarta Validation, JUnit 5, Mockito, Testcontainers.

## 4.3 Princípios da Arquitetura Backend

SRP, DIP, baixo acoplamento, alta coesão, separação entre domínio e infraestrutura, injeção de dependência e código limpo.

## 4.4 Organização Geral do Projeto

Abordagem orientada a funcionalidades (feature-first) com diretórios específicos como shared/, security/, config/, e módulos para cada entidade do negócio.

## 4.5 Estrutura Interna de Cada Módulo

Cada módulo (ex: territories/) divide-se em: application/ (dto, mapper, service, usecase), domain/ (entity, repository, service, event), infrastructure/ (persistence, integration) e interfaces/ (controller, exception).

## 4.6 Camada de Interfaces

Contém os Controllers (ex: AuthController, TerritoryController) encarregados de receber requisições HTTP, validar dados básicos e acionar os casos de uso sem implementar regras de negócio diretamente.

## 4.7 Camada de Aplicação

Implementa os casos de uso coordenando entidades, controlando transações e acionando serviços de domínio.

## 4.8 Camada de Domínio

Núcleo puro da aplicação, contendo entidades, agregados e contratos de repositórios sem dependência de frameworks.

## 4.9 Camada de Infraestrutura



Implementa detalhes técnicos como persistência JPA, geração de tokens JWT, envio de e-mails e logs.

## **4.11 DTOs (Data Transfer Objects) e Mapeamento**

Isolam o domínio usando estruturas de Request/Response. O MapStruct realiza a conversão automática entre entidades e DTOs.

## **4.13 Persistência e Tratamento de Exceções**

Utiliza Spring Data JPA com repositórios dedicados para cada agregado. Exceções são centralizadas em um GlobalExceptionHandler.

## **4.15 Validações e Auditoria**

Validações em múltiplos níveis (Bean Validation e invariantes). Operações críticas geram registros automáticos contendo usuário, data/hora e operação afetada.

## **4.17 Estratégia de Testes**

Testes Unitários (Mockito), Testes de Integração (Testcontainers com PostgreSQL) e Testes de Segurança (filtros e permissões).

# **Capítulo 5 – Arquitetura Frontend**

## **5.1 Objetivo**

Definir a arquitetura do frontend do GTP para fornecer uma interface intuitiva, gerenciar o estado e consumir a API REST.

## **5.2 Stack Tecnológica**

TypeScript, React 19, Vite, Tailwind CSS, Shadcn UI, Zustand, TanStack Query, React Router, React Hook Form, Zod, Axios, Lucide React.

## **5.3 Princípios Arquiteturais**

Feature First, separação entre lógica e apresentação, reutilização de componentes, responsividade e tipagem forte com TypeScript.

## **5.4 Organização Geral do Projeto**

Estrutura padrão com diretórios para components/, features/, hooks/, layouts/, services/ e stores/.

## **5.7 Layouts e Gerenciamento de Estado**

Layouts dedicados (Público, Administrativo, Operacional). O estado global é gerenciado de forma simplificada utilizando Zustand (AuthStore, TerritoryStore, etc.).

## 5.9 Comunicação com a API e Formulários

Axios para requisições HTTP e TanStack Query para cache e sincronização. Formulários gerenciados com React Hook Form e validados via Zod.

## 5.12 Design System e Responsividade

Uso de componentes do Shadcn UI com abordagem mobile-first atendendo smartphones, tablets, notebooks e desktops com padrões estritos de acessibilidade.

# Capítulo 6 – Segurança da Arquitetura

## 6.1 Objetivo e Princípios

Proteger informações sensíveis com base nos princípios de Security by Design, menor privilégio, negação por padrão e defesa em profundidade.

## 6.3 Autenticação e Autorização (RBAC)

Autenticação baseada em tokens JWT. O controle de acesso utiliza perfis bem delimitados:

- **Desenvolvedor Geral:** Administração técnica e evolução do sistema.
- **Administrador Geral:** Administração global da Congregação.
- **Servo de Territórios:** Gestão de territórios, reservas e devoluções.
- **Servo de Publicações:** Controle de estoque, pedidos e entregas.
- **Publicador:** Solicitação de territórios e publicações.

## 6.7 Proteção de Dados e Proteção contra Vulnerabilidades

Senhas criptografadas com BCrypt (nunca registradas em logs ou retornadas). Consultas parametrizadas via JPA contra SQL Injection, políticas estritas de CORS e uso obrigatório de HTTPS.

# Capítulo 7 – Conclusão da Arquitetura e Diretrizes para Implementação

## 7.1 Conclusão

O Documento 05 estabelece as fundações técnicas sólidas do GTP através de um DDD pragmático e Clean Architecture. A escolha de tecnologias maduras (Java 21, Spring Boot 3, React 19, TypeScript e PostgreSQL 17) garante uma solução robusta, modular, escalável e segura, perfeitamente alinhada às necessidades do negócio e preparada para evoluções futuras.